

CORS и безопасность HTTP-взаимодействия

Same-Origin Policy, preflight, credentials, cookies и CSRF

Результаты занятия

- ✓ Объяснять Same-Origin Policy
- ✓ Определять origin по scheme, host и port
- ✓ Объяснять назначение CORS
- ✓ Различать simple request и preflight request
- ✓ Понимать роль OPTIONS
- ✓ Настраивать cors middleware в Express
- ✓ Различать Bearer token и cookie credentials
- ✓ Использовать credentials: include для cross-origin cookie requests
- ✓ Понимать HttpOnly, Secure и SameSite
- ✓ Объяснять риск CSRF и способы его снижения

Учебные вопросы

- 1 Same-Origin Policy** Почему браузер ограничивает доступ JavaScript к другому origin
- 2 CORS и preflight** Origin, Access-Control-* headers и OPTIONS
- 3 Настройка Express** cors middleware и разрешённые origins
- 4 Credentials и cookies** credentials: include, HttpOnly, Secure и SameSite
- 5 CSRF и безопасная конфигурация** Почему cookie-auth требует дополнительных мер защиты



Учебный вопрос 1. Same-Origin Policy

Браузер ограничивает взаимодействие между разными origins

Что такое origin

- Origin определяется тройкой: scheme + host + port.
- `http://localhost:5500` и `http://localhost:3000` — разные origins из-за port.
- `http://example.com` и `https://example.com` — разные origins из-за scheme.
- `https://api.example.com` и `https://shop.example.com` — разные origins из-за host.
- Path не входит в определение origin.

Сравнение origins

Frontend	Backend	Same origin?	Причина
http://localhost:5500	http://localhost:5500	Да	Совпадают scheme, host и port
http://localhost:5500	http://localhost:3000	Нет	Разный port
http://shop.local	https://shop.local	Нет	Разный scheme
https://shop.example.com	https://api.example.com	Нет	Разный host

Same-Origin Policy

- Same-Origin Policy ограничивает доступ script-кода одного origin к данным другого origin.
- Это важная базовая browser security boundary.
- Без ограничений любой сайт мог бы читать чувствительные ответы другого сайта от имени пользователя.
- SOP применяется браузером.
- CORS позволяет серверу осознанно открыть доступ определённым origins.

Same-Origin Policy: frontend :5500 и backend :3000 — разные origins

Один и тот же сайт (scheme + host + port) — это один origin. Если отличается хотя бы одна часть — это другой origin.



Origin = scheme + host + port

Frontend origin
http://localhost:5500

≠

Backend origin
http://localhost:3000



Здесь scheme (http) и host (localhost) совпадают, но port разный (5500 ≠ 3000), поэтому это разные origins. Без CORS браузер блокирует доступ JavaScript к response.

CORS не защищает API от всех клиентов

- CORS проверяется браузером.
- curl, Postman и server-to-server clients не обязаны соблюдать browser CORS policy.
- CORS не заменяет Authentication.
- CORS не заменяет Authorization.
- Backend всё равно должен проверять JWT, role, ownership и validation.

Что происходит без CORS

- Frontend JavaScript отправляет cross-origin request.
- Backend может фактически обработать запрос.
- Но если CORS headers не разрешают origin, браузер не отдаёт response JavaScript-коду.
- В DevTools появляется CORS error.



Учебный вопрос 2. CORS и preflight

Сервер сообщает браузеру, каким origins разрешено читать ответы

Cross-Origin Resource Sharing

- CORS — механизм HTTP headers, позволяющий серверу разрешить browser JavaScript с другого origin читать его responses.
- Browser отправляет Origin header.
- Server возвращает Access-Control-Allow-Origin.
- Browser сравнивает policy и решает, передавать ли response JavaScript-коду.
- Разрешать следует минимально необходимое число origins.

Пример CORS response header

HTTP

```
Access-Control-Allow-Origin: http://localhost:5500
```

Simple и preflighted requests

- Некоторые cross-origin requests браузер отправляет сразу.
- Другие требуют предварительного OPTIONS request.
- Preflight спрашивает сервер: разрешены ли method и headers будущего запроса?
- После успешного preflight браузер отправляет actual request.

CORS Preflight: OPTIONS → CORS response → Actual request

Для «непростых» запросов (POST + application/json + Authorization) браузер сначала отправляет preflight (OPTIONS), проверяет CORS headers, и только затем выполняет настоящий запрос.



Internet Shop
Learn. Build. Deploy.



Браузер (Frontend)

http://localhost:5500



Это не простой запрос, так как:

- метод POST (не GET/HEAD/POST с простыми заголовками)
- Content-Type: application/json
- есть заголовок Authorization

→ Браузер отправит preflight (OPTIONS)



Что такое preflight?

- Браузер автоматически отправляет OPTIONS для проверки, разрешён ли кросс-доменный запрос.
- Используется для «непростых» запросов (методы, заголовки, типы контента).
- Если сервер не разрешает запрос — настоящий запрос не отправляется.

1

Preflight request (OPTIONS)

Браузер отправляет OPTIONS, чтобы узнать, разрешён ли запрос.

OPTIONS /api/orders HTTP/1.1

Origin: http://localhost:5500
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type, authorization

2

CORS response

Сервер отвечает CORS headers. Если всё разрешено — браузер выполняет настоящий запрос.

HTTP/1.1 204 No Content

Access-Control-Allow-Origin: http://localhost:5500
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: content-type, authorization
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 600

3

Actual request (POST)

После успешного preflight браузер отправляет настоящий запрос.

POST /api/orders HTTP/1.1

Origin: http://localhost:5500
Content-Type: application/json
Authorization: Bearer <token>
{ "productId": 1, "quantity": 1 }

HTTP/1.1 201 Created

Access-Control-Allow-Origin: http://localhost:5500
Access-Control-Allow-Credentials: true
Content-Type: application/json
{ "id": 123, "status": "created" }



Сервер (Backend)

http://localhost:3000

Express.js + cors middleware

```
import express from 'express';
import cors from 'cors';

const app = express();

app.use(cors({
  origin: 'http://localhost:5500',
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true
}));

app.use(express.json());

app.post('/api/orders', (req, res) => {
  // обработка заказа
  res.status(201).json({ id: 123, status: 'created' });
});
```



Сервер разрешает запросы с **http://localhost:5500** и возвращает необходимые CORS headers.



Если **origin** не разрешён, сервер не возвращает CORS headers, и браузер блокирует доступ к response (даже если сервер вернул 200 OK).



Важно

- CORS — это механизм безопасности браузера.
- Postman, curl и серверные клиенты не используют CORS.
- Access-Control-Max-Age позволяет кэшировать результат preflight.



Простые запросы (без preflight)

- GET, HEAD, POST
- Только простые заголовки (например, Content-Type: text/plain, application/x-www-form-urlencoded, multipart/form-data)
- Без Authorization и других кастомных заголовков.

Пример preflight request

HTTP

```
OPTIONS /api/orders HTTP/1.1  
Origin: http://localhost:5500  
Access-Control-Request-Method: POST  
Access-Control-Request-Headers: authorization, content-type
```

Пример preflight response

HTTP

HTTP/1.1 204 No Content

Access-Control-Allow-Origin: http://localhost:5500

Access-Control-Allow-Methods: GET,POST,PATCH,DELETE

Access-Control-Allow-Headers: Content-Type,Authorization

Почему Bearer + JSON часто приводит к preflight

- Authorization — нестандартный для CORS-safelisted request header.
- application/json не относится к safelisted Content-Type.
- Поэтому browser может сначала выполнить OPTIONS.
- Frontend обычно не пишет OPTIONS вручную — браузер делает это сам.

Access-Control-* headers

- Access-Control-Allow-Origin — какие origins разрешены.
- Access-Control-Allow-Methods — какие HTTP methods разрешены.
- Access-Control-Allow-Headers — какие request headers разрешены.
- Access-Control-Allow-Credentials — можно ли делать credentialed cross-origin requests.
- Access-Control-Max-Age — сколько browser может кэшировать результат preflight.



Учебный вопрос 3. Настройка CORS в Express

В Express обычно используют middleware cors

Установка cors

BASH

```
npm install cors
```

Минимальная настройка для frontend :5500

JAVASCRIPT

```
import cors from "cors";

app.use(cors({
  origin: "http://localhost:5500"
}));
```

Что важно

- Разрешён только конкретный frontend origin.
- Middleware добавляет CORS response headers.
- Browser применяет эту policy.

Express cors middleware: Origin → policy → Access-Control-Allow-Origin



Internet Shop

Learn. Build. Deploy.

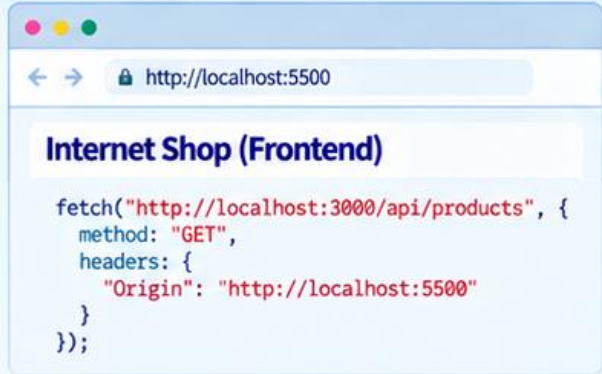
Браузер отправляет заголовок Origin. Express cors сравнивает его с разрешённой политикой.

Если origin разрешён — сервер возвращает соответствующий заголовок, иначе браузер блокирует ответ.



Браузер (Frontend)

http://localhost:5500



Браузер автоматически добавляет заголовок Origin при кросс-доменном запросе.



Заголовок Origin

Содержит источник запроса (scheme + host + port).

Пример: Origin: http://localhost:5500
Отправляется автоматически для fetch, XHR и некоторых других запросов.

1. Запрос
Origin: http://localhost:5500



Express приложение (Backend)

http://localhost:3000

Настройка cors middleware

```
import express from 'express';
import cors from 'cors';

const app = express();

const allowedOrigins = [
  'http://localhost:5500',
  'https://shop.example.com'
];

app.use(cors({
  origin: (origin, callback) => {
    if (!origin) return callback(null, true); // например, для Postman
    if (allowedOrigins.includes(origin)) {
      return callback(null, true);
    } else {
      return callback(new Error('Not allowed by CORS'));
    }
  },
  credentials: true
}));
```



Политика (policy)

Разрешённые origins:

- ✓ http://localhost:5500
- ✓ https://shop.example.com
- ✓ https://admin.example.com

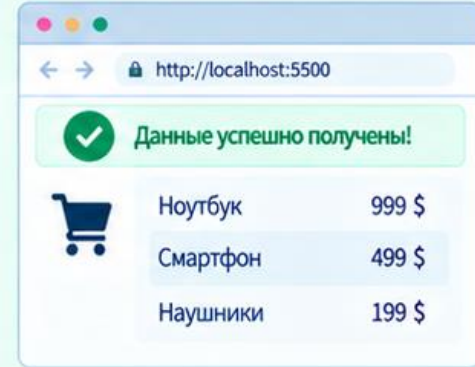


Разрешённый origin (успешный запрос)

2. Ответ сервера (CORS headers)

HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:5500
Access-Control-Allow-Credentials: true
Content-Type: application/json
{ "products": [...] }

3. Браузер получает ответ и даёт доступ к данным



Другой origin (заблокированный)



1. Запрос
Origin: http://evil.com

Origin не разрешён политикой



cors middleware не добавляет Access-Control-Allow-Origin для этого origin.

2. Ответ сервера

HTTP/1.1 200 OK
Content-Type: application/json
{ "products": [...] }

3. Браузер блокирует доступ к response



Важно помнить

- CORS — это механизм безопасности браузера, а не сервера.
- Сервер всегда может вернуть ответ, но браузер решает, может ли JavaScript получить к нему доступ.
- Postman, curl и серверные клиенты не используют CORS.



Когда разрешать origin

- Используйте белый список (whitelist).
- В продакшене указывайте конкретные origins (без *).
- Для cookie (credentials: true) нельзя использовать *.
- Учитывайте разные окружения (dev, staging, prod).



Чего не делать

- Не использовать Access-Control-Allow-Origin: * при credentials: true.
- Не разрешать все источники без необходимости.
- Не полагаться на CORS как на единственную защиту API.
- Всегда дополнительно проверяйте аутентификацию и права.



Дополнительная защита

- Проверяйте Authorization / session.
- Используйте CSRF-защиту (для cookie-based auth).
- Настраивайте правильные CORS headers.
- Ограничивайте методы и заголовки.
- Логируйте отклонённые запросы.

Не используйте origin: * без необходимости

- Для публичного non-credentialed API wildcard может быть допустим.
- Для приватного приложения обычно лучше перечислить доверенные origins.
- Минимальная CORS policy уменьшает поверхность нежелательного cross-origin чтения.
- Credentialed requests нельзя сочетать с Access-Control-Allow-Origin: *.

Явная CORS policy

JAVASCRIPT

```
app.use(cors({  
  origin: "http://localhost:5500",  
  methods: ["GET", "POST", "PATCH", "DELETE"],  
  allowedHeaders: ["Content-Type", "Authorization"]  
}));
```

Что важно

- methods ограничивает разрешённые HTTP methods.
- allowedHeaders явно разрешает JSON и Bearer JWT.
- OPTIONS обрабатывается cors middleware.

Разные origins для development и production

- В development frontend может работать на localhost:5500.
- В production frontend может быть https://shop.example.com.
- Не хардкойте production policy в нескольких файлах.
- Разрешённый frontend origin удобно задавать через environment.

FRONTEND_ORIGIN из environment

JAVASCRIPT

```
app.use(cors({  
  origin: process.env.FRONTEND_ORIGIN,  
  methods: ["GET", "POST", "PATCH", "DELETE"],  
  allowedHeaders: ["Content-Type", "Authorization"]  
}));
```

Что важно

- Development и production используют разные .env.
- Policy остаётся централизованной.

CORS error ≠ ошибка Express Controller

- CORS failure часто происходит на уровне browser policy.
- Controller может вообще не получить actual request, если preflight запрещён.
- Или backend может вернуть response, но browser скроет его от JavaScript.
- Всегда проверяйте DevTools Network и Console.



Учебный вопрос 4. Credentials и cookies

Cookie-based Authentication меняет требования к CORS

Что такое credentials в Fetch

- Для cross-origin fetch cookies по умолчанию не отправляются.
- Чтобы запрос включал credentials, используют credentials: include.
- Server должен разрешить credentialed CORS.
- Browser дополнительно применяет cookie attributes.

Cross-origin fetch c cookie

JAVASCRIPT

```
const response = await fetch(
  "http://localhost:3000/api/auth/me",
  {
    credentials: "include"
  }
);
```

Что важно

- Browser просит включить cookies.
- Server должен вернуть Access-Control-Allow-Credentials: true.
- Access-Control-Allow-Origin должен быть конкретным origin, а не *.

Credentialed CORS: credentials include + explicit origin + allow credentials

Полный поток кросс-доменных запросов с учётом учетных данных (cookies)



Frontend (Browser)

http://localhost:5500



http://localhost:5500

```
fetch("http://localhost:3000/api/auth/me", {
  credentials: "include"
})
.then(res => res.json())
.then(data => console.log(data));
```



Браузер отправляет cookies в кросс-доменных запросах только если указано `credentials: "include"`.

Кросс-доменный запрос

GET /api/auth/me
Origin: http://localhost:5500
Cookie: session=abc123...



Cookies отправляются в запросе



Express API (Backend)

http://localhost:3000



Настройка CORS (Express)

```
app.use(cors({
  origin: 'http://localhost:5500',
  credentials: true
}));
```

Ответ сервера (заголовки)

HTTP/1.1 200 OK
Access-Control-Allow-Origin:
`http://localhost:5500`
Access-Control-Allow-Credentials: `true`



Origin должен быть указан явно и совпадать с origin фронтенда.



Браузер получает ответ

http://localhost:5500



http://localhost:5500

```
// Данные доступны JavaScript
{
  "user": {
    "id": 1,
    "email": "user@example.com"
  }
}
```



Доступ разрешён

Ответ доступен JavaScript, можно работать с данными.



* + credentials — нельзя

Неверная конфигурация CORS (не работает с учетными данными):

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
```



Использовать wildcard (*) вместе с credentials нельзя.

Когда запрос включает учетные данные (cookies), браузер отклоняет ответ, если Access-Control-Allow-Origin: *. Необходимо указать конкретный origin (например, http://localhost:5500).



Итоги: 3 ключевых условия для credentialed CORS

1

credentials: "include"

Браузер отправляет cookies в кросс-доменных запросах.

2

Access-Control-Allow-Origin должен быть конкретным

Укажите явный origin (например, http://localhost:5500).

3

Access-Control-Allow-Credentials: true

Обязательно разрешите использование учетных данных на бэкенде.

Express CORS для cookie-based frontend

JAVASCRIPT

```
app.use(cors({  
  origin: "http://localhost:5500",  
  credentials: true,  
  methods: ["GET", "POST", "PATCH", "DELETE"],  
  allowedHeaders: ["Content-Type"]  
}));
```

Что важно

- `credentials: true` добавляет `Access-Control-Allow-Credentials: true`.
- `origin` задаётся явно.
- Wildcard `*` здесь не используется.

Cookie attributes

Attribute	Назначение	Пример
HttpOnly	Запрещает JavaScript читать cookie	HttpOnly
Secure	Отправлять cookie только по HTTPS	Secure
SameSite	Ограничивает cross-site отправку cookie	Lax / Strict / None
Path	Ограничивает URL path	Path=/
Max-Age/Expires	Определяет срок жизни	Max-Age=900

HttpOnly

- HttpOnly cookie недоступна через `document.cookie`.
- Это снижает риск прямой кражи `session credential` через XSS.
- Но XSS всё ещё может инициировать `requests` от имени пользователя.
- HttpOnly — важная защита, но не единственная.

Secure и SameSite

- Secure требует HTTPS для отправки cookie.
- SameSite=Strict сильнее ограничивает cross-site отправку.
- SameSite=Lax допускает некоторые top-level safe navigations.
- SameSite=None требует Secure.
- SameSite помогает против CSRF, но не всегда заменяет полноценную CSRF-защиту.



Учебный вопрос 5. CSRF и безопасная конфигурация

Автоматическая отправка cookie создаёт отдельный класс угроз

Что такое CSRF

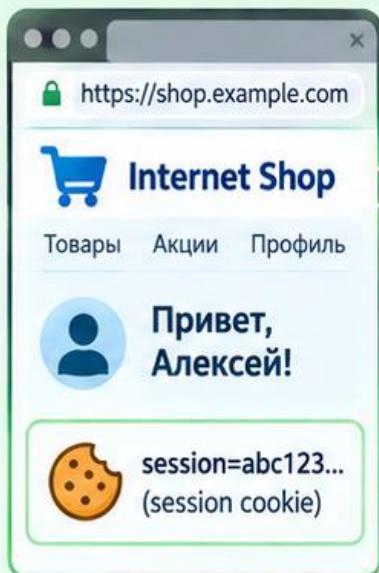
- Cross-Site Request Forgery — атака, при которой злоумышленник заставляет browser аутентифицированного пользователя отправить нежелательный state-changing request.
- Проблема особенно актуальна для cookie-based Authentication.
- Browser может автоматически приложить session cookie.
- Атакующему не обязательно знать сам cookie.
- Поэтому state-changing endpoints требуют дополнительной защиты.

Упрощённый CSRF-сценарий

- 1 User logged in** Browser имеет session cookie магазина.
- 2 Открывает malicious site** Пользователь посещает страницу атакующего.
- 3 Сайт инициирует request** Browser формирует запрос к магазину.
- 4 Cookie может приложиться автоматически** Backend видит authenticated request.
- 5 Без CSRF-защиты** State-changing action может выполняться.

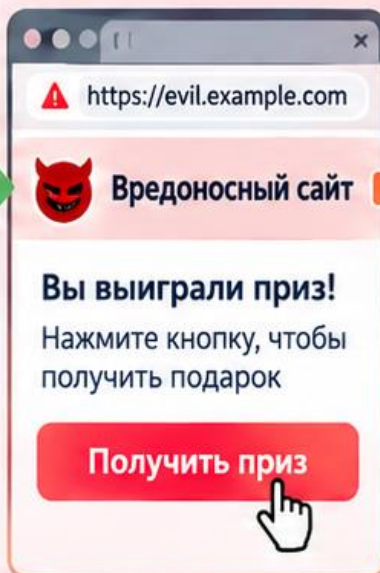
CSRF: вредоносный сайт → браузер с session cookie → защищённое действие

1. Пользователь уже вошёл в Internet Shop



Пользователь уже вошёл в Internet Shop

2. Пользователь открывает вредоносный сайт



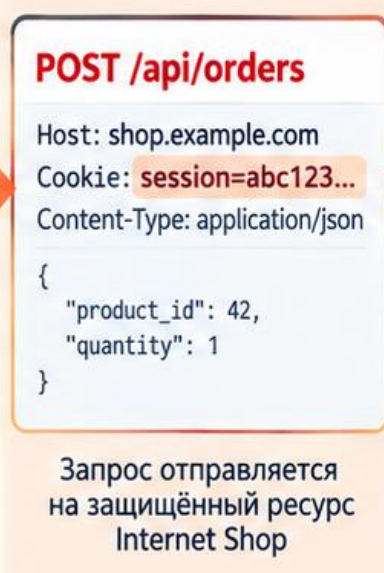
Пользователь открывает вредоносный сайт (например, по ссылке в письме, мессенджере и т.п.)

3. Браузер с session cookie



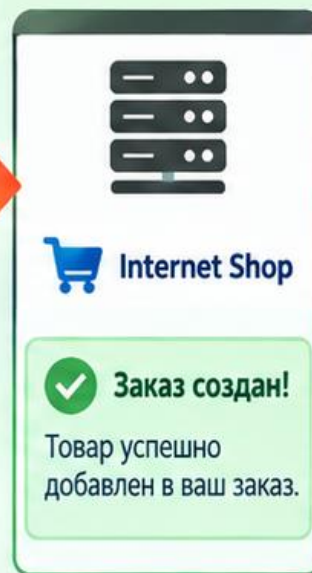
Браузер автоматически добавляет session cookie к запросам на домен Internet Shop

4. state-changing request к Internet Shop



Без дополнительной защиты запрос может быть выполнен от имени пользователя.

5. Защищённое действие



Например: создание заказа, изменение email, добавление адреса и т.п.

✓ Меры защиты

SameSite
ограничивает отправку cookie в cross-site сценариях.

CSRF token
сервер проверяет специальный токен для state-changing запросов.

Origin verification
проверка Origin/Referer для чувствительных запросов.

Правильные HTTP methods
не выполнять изменение состояния через GET; использовать POST/PATCH/DELETE.



Главная идея: cookie-based authentication требует защиты от CSRF.

Защита от CSRF

- Не изменять server state через GET.
- Использовать SameSite как defense in depth.
- Проверять Origin/Referer для state-changing requests.
- Использовать CSRF token или double-submit pattern, когда архитектура этого требует.
- Использовать HTTPS и Secure cookies.

Bearer JWT и Cookie session

Authorization: Bearer

- JavaScript явно добавляет token
- Browser не прикладывает token автоматически
- CSRF риск обычно ниже
- XSS может украсть JS-доступный token
- CORS всё равно нужен для cross-origin browser access

HttpOnly Cookie

- Browser отправляет cookie автоматически
- JavaScript не читает HttpOnly credential
- Снижает риск прямой кражи token через XSS
- Требуется внимания к CSRF
- Credentialed CORS требует explicit origin

Безопасная HTTP-конфигурация

- ✓ Разрешать только нужные origins
- ✓ Не считать CORS Authentication
- ✓ Для credentials не использовать Access-Control-Allow-Origin: *
- ✓ Использовать HTTPS в production
- ✓ Использовать HttpOnly и Secure для session cookies
- ✓ Выбирать SameSite осознанно
- ✓ Не выполнять state changes через GET
- ✓ Защищать cookie-auth от CSRF
- ✓ Не доверять frontend как границе безопасности
- ✓ Проверять Authentication и Authorization на backend

Мини-практика: CORS между :5500 и :3000

Настройте frontend на `http://localhost:5500` и Express API на `http://localhost:3000`, затем исследуйте CORS и preflight.

Задание

- Запустить frontend и backend на разных ports
- Убедиться, что origins различаются
- Временно отключить cors middleware и увидеть CORS error
- Установить cors
- Разрешить только `http://localhost:5500`
- Выполнить GET `/api/products`
- Выполнить POST с JSON и Authorization header
- Найти OPTIONS preflight в DevTools
- Проверить Access-Control-* headers
- Проверить блокировку другого origin

Критерии проверки

- Frontend :5500 читает ответы API :3000
- Другой origin не получает разрешение CORS
- Студент находит OPTIONS preflight
- Студент объясняет, почему CORS не заменяет JWT
- CORS policy не использует wildcard для private API

Как диагностировать CORS

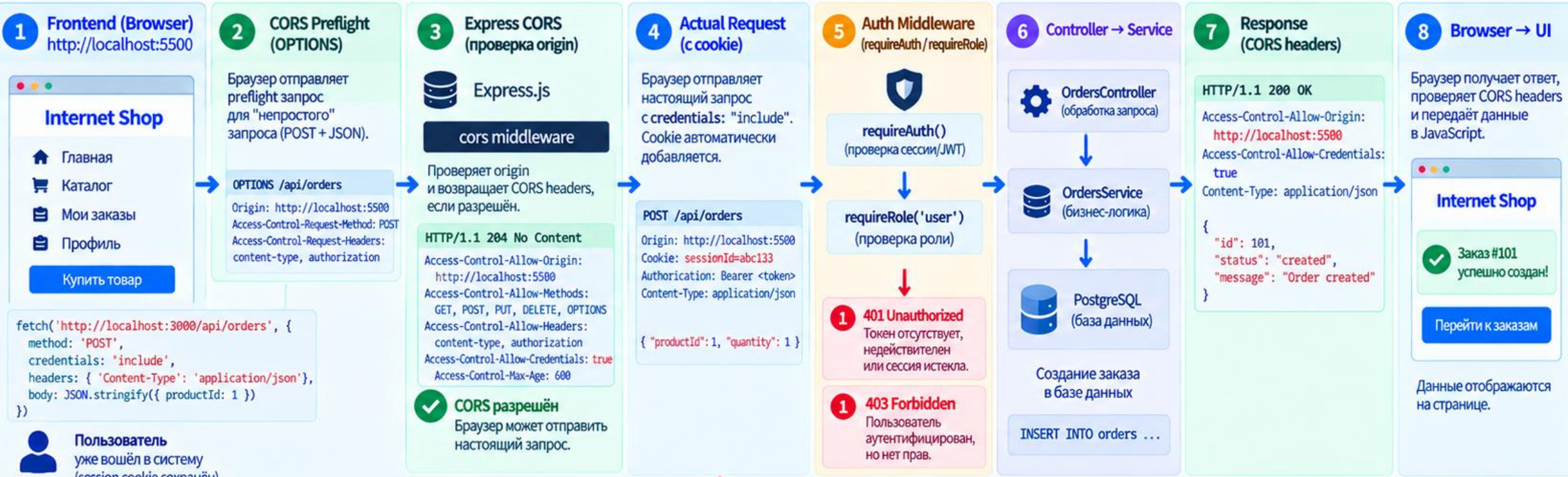
- Откройте Browser DevTools → Network.
- Ищите OPTIONS request перед actual request.
- Проверьте Origin request header.
- Проверьте Access-Control-Allow-Origin в response.
- Проверьте allowed methods/headers.
- Читайте Console: browser обычно объясняет причину блокировки.

Полный secure cross-origin flow — Internet Shop

От браузера до базы данных и обратно. CORS, аутентификация, авторизация и защита.



Internet Shop
Learn. Build. Deploy.



Cookie и CORS

- ✓ fetch(..., { credentials: "include" })
- ✓ На сервере: Access-Control-Allow-Credentials: true
- ✓ Access-Control-Allow-Origin: конкретный origin (не *)
- ✓ Cookie: HttpOnly, Secure, SameSite (Lax/Strict)

Защита от CSRF

- Используйте SameSite=Lax/Strict для session cookies
- Для state-changing запросов — CSRF token (double submit или synchronizer token)
- Проверяйте заголовок Origin/Referer
- Используйте правильные HTTP методы (POST/PUT/DELETE), не GET

Дополнительные рекомендации

- Настраивайте CORS для конкретных origin (whitelist)
- Ограничивайте разрешённые методы и заголовки
- Логируйте отклонённые запросы
- Используйте короткий срок жизни токенов
- Проверьте роли на уровне middleware

Результат

Безопасное HTTP-взаимодействие между frontend и backend с учётом CORS, аутентификации, авторизации и защиты от CSRF.

Контрольные вопросы

1. Из каких трёх частей состоит origin?
2. Почему localhost:5500 и localhost:3000 — разные origins?
3. Что делает Same-Origin Policy?
4. Что делает CORS?
5. Почему CORS не защищает API от Postman?
6. Для чего browser отправляет OPTIONS preflight?
7. Какие задачи решают Access-Control-Allow-Origin и Access-Control-Allow-Headers?
8. Почему credentialed CORS нельзя использовать с origin *?
9. Для чего нужны HttpOnly, Secure и SameSite?
10. Почему cookie-based Authentication требует защиты от CSRF?

Итоги занятия

- 1 Origin определяется scheme, host и port.
- 2 Same-Origin Policy ограничивает cross-origin доступ browser JavaScript.
- 3 CORS позволяет серверу явно разрешать доверенные origins.
- 4 Preflight OPTIONS проверяет возможность будущего cross-origin request.
- 5 Express cors middleware формирует необходимые CORS headers.
- 6 CORS является browser policy и не заменяет Authentication/Authorization.
- 7 Credentialed fetch использует credentials: include.
- 8 Для credentialed CORS нужен explicit origin и Access-Control-Allow-Credentials.
- 9 HttpOnly, Secure и SameSite усиливают cookie security.
- 10 Cookie-based Authentication требует продуманной CSRF-защиты.